



哈尔滨工业大学 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

课程报告

开课学期: 2023 夏季

课程名称: 计算机设计与实践

项目名称: 基于 miniRV 的 SoC 设计

项目类型: 综合设计型

课程学时: 56 地点: T2506

学生班级: 8

学生学号: 210110812

学生姓名: 李春阳

评阅教师: _____

报告成绩: _____

实验与创新实践教育中心制

2023 年 7 月

设计概述（罗列出所有实现的指令，以及单周期/流水线 CPU 频率）

实现指令：单周期与流水线 CPU 均实现 24 条基础指令

指令描述：

助记符	使用语法	操作及解释
R 型		
add	add rd,rs1,rs2	$(rd) \leftarrow (rs1) + (rs2)$
sub	sub rd,rs1,rs2	$(rd) \leftarrow (rs1) - (rs2)$
and	and rd,rs1,rs2	$(rd) \leftarrow (rs1) \& (rs2)$
or	or rd,rs1,rs2	$(rd) \leftarrow (rs1) (rs2)$
xor	xor rd,rs1,rs2	$(rd) \leftarrow (rs1) \wedge (rs2)$
sll	sll rd,rs1,rs2	$(rd) \leftarrow (rs1) \ll (rs2)$
srl	srl rd,rs1,rs2	$(rd) \leftarrow (rs1) \gg (rs2)$, 逻辑右移
sra	sra rd,rs1,rs2	$(rd) \leftarrow (rs1) \gg (rs2)$, 算术右移
I 型		
addi	addi rd,rs1,imm	$(rd) \leftarrow (rs1) + \text{sext}(\text{imm})$
andi	andi rd,rs1,imm	$(rd) \leftarrow (rs1) \& \text{sext}(\text{imm})$
ori	ori rd,rs1,imm	$(rd) \leftarrow (rs1) \text{sext}(\text{imm})$
xori	xori rd,rs1,imm	$(rd) \leftarrow (rs1) \wedge \text{sext}(\text{imm})$
slli	slli rd,rs1,imm	$(rd) \leftarrow (rs1) \ll \text{shamt}$
srli	srli rd,rs1,imm	$(rd) \leftarrow (rs1) \gg \text{shamt}$, 逻辑右移
srai	srai rd,rs1,imm	$(rd) \leftarrow (rs1) \gg \text{shamt}$, 算术右移
lw	lw rd,offset(rs1)	$(rd) \leftarrow \text{sext}(\text{Mem}[(rs1) + \text{sext}(\text{offset})][31:0])$
jalr	jalr rd,offset(rs1)	$t \leftarrow (pc) + 4; (pc) \leftarrow ((rs1) + \text{sext}(\text{offset})) \& \sim 1; (rd) \leftarrow t$
S 型		
sw	sw rs2,offset(rs1)	$\text{Mem}[(rs1) + \text{sext}(\text{offset})] \leftarrow (rs2)[31:0]$
B 型		
beq	beq rs1,rs2,offset	if $((rs1) = (rs2)) (pc) \leftarrow (pc) + \text{sext}(\text{offset})$
bne	bne rs1,rs2,offset	if $((rs1) \neq (rs2)) (pc) \leftarrow (pc) + \text{sext}(\text{offset})$
blt	blt rs1,rs2,offset	if $((rs1) < (rs2)) (pc) \leftarrow (pc) + \text{sext}(\text{offset})$, 有符号比较
bge	bge rs1,rs2,offset	if $((rs1) \geq (rs2)) (pc) \leftarrow (pc) + \text{sext}(\text{offset})$, 有符号比较
U 型		
lui	lui rd,imm	$(rd) \leftarrow \text{sext}(\text{imm}[31:12]) \ll 12$
J 型		
jal	jal rd,offset	$(rd) \leftarrow (pc) + 4; (pc) \leftarrow (pc) + \text{sext}(\text{offset})$

时钟频率：单周期 CPU 频率为 25MHZ，流水线 CPU 频率为 100MHZ

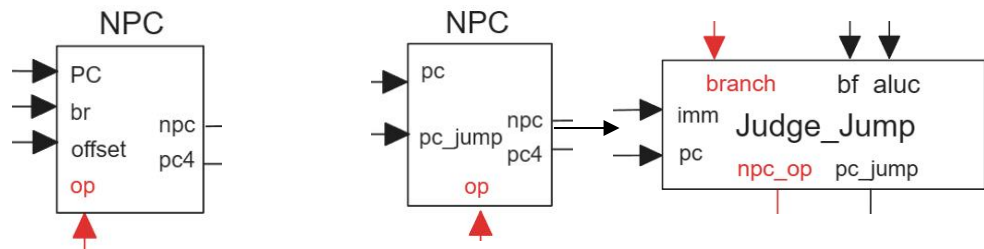
设计的主要特色（除基本要求以外的设计）

1. 实现分支预测功能：

在流水线 CPU 中采用静态分支预测，默认不进行跳转。分支检测位于 EX 阶段，因此当预测错误（即需要进行跳转）时，需要插入两个空周期。

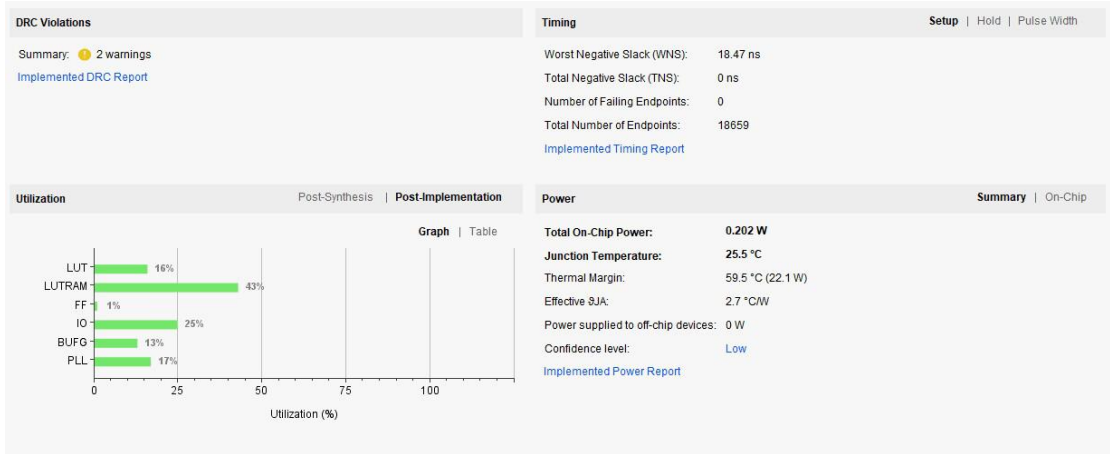
2. 将跳转类指令归类以减少硬件

在 24 条基本指令中仅 B 型指令、jalr 指令、jal 指令存在跳转。为节省硬件资源，在单周期的基础上重新设计 NPC 部件并在 EX 阶段单独设置“跳转”模块计算上述三种需要跳转的指令所跳转到的地址并传回 NPC 部件。

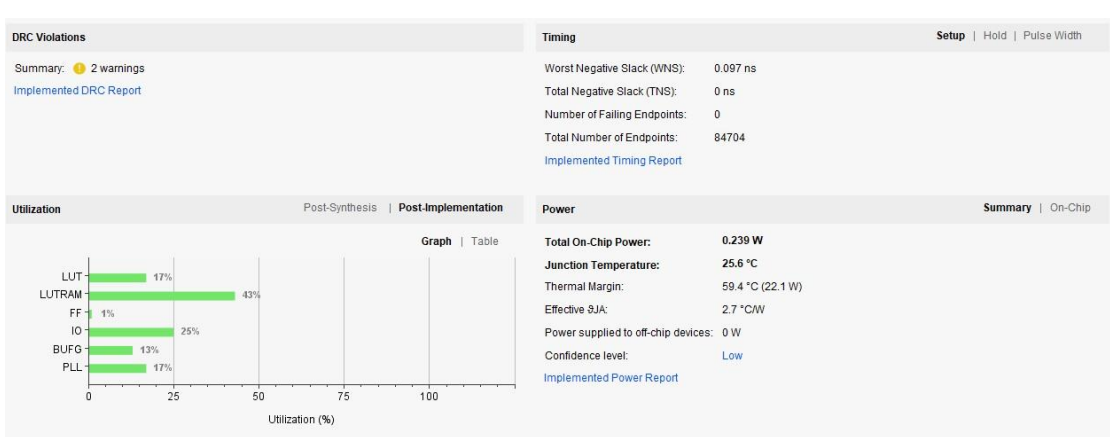


资源使用、功耗数据截图（Post Implementation；含单周期、流水线 2 个截图）

单周期：



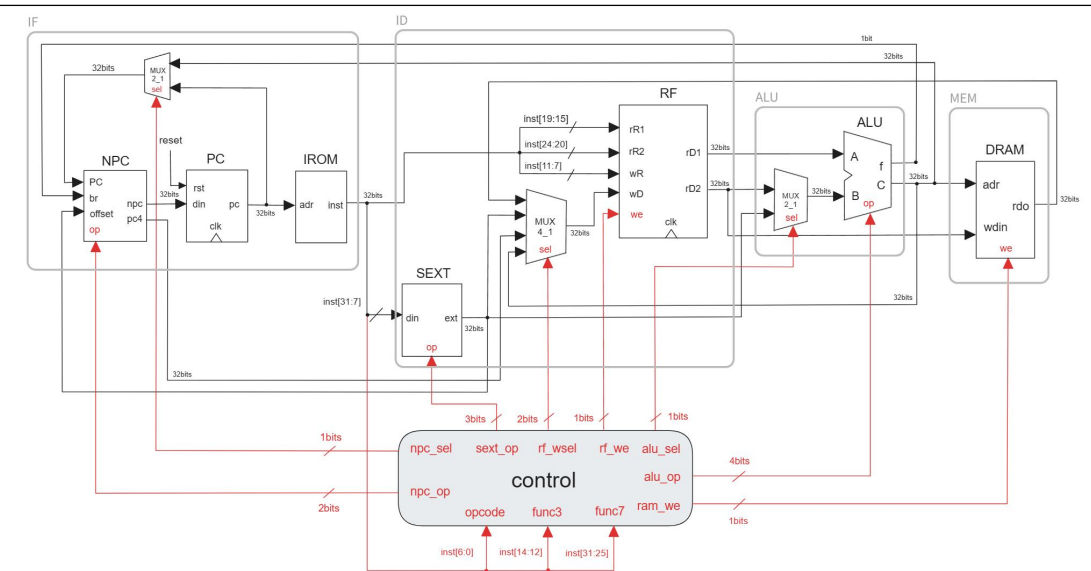
流水线：



1 单周期 CPU 设计与实现

1.1 单周期 CPU 数据通路设计

要求：贴出完整的单周期数据通路图，无需画出模块内的具体逻辑，但要标出模块的接口信号名、模块之间信号线的信号名和位宽，并用文字阐述各模块的功能。



模块说明：

1. 取指模块

- PC：存储当前指令地址的 32 位寄存器，指令执行时负责更新当前 PC 值（时序逻辑）
- NPC：得到下一条指令的 PC 值（组合逻辑）
- IROM：指令存储器，根据 PC 值得到对应的指令
- MUX：多路选择器，选择 NPC.pc 的值（组合逻辑）

2. 译码模块

- RF：寄存器堆，可根据寄存器编号读出源寄存器数据或将数据写入目标寄存器（异步读（组合逻辑）、同步写（时序逻辑））
- SEXT：立即数扩展单元，根据指令中立即数的格式进行扩展（组合逻辑）
- MUX：多路选择器，选择写回寄存器堆的数据

3. 执行模块

- ALU：运算器，根据控制信号对输入数据执行加、减、与、或、异或、移位、比较等运算（组合逻辑）
- MUX：多路选择器，选择输入 ALU.B 端口的数据

4. 存储模块

- DRAM：数据存储器，可读可写，用于存储数据

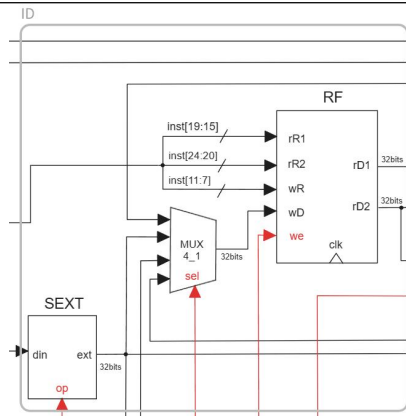
5. 写回模块

6. 控制模块

- control：控制器，根据指令中 opcode、func3、func7 的值得到各个器件的控制信号

1.2 单周期 CPU 模块详细设计

要求：以表格的形式列出各个部件的接口信号、位宽、功能描述等，并结合图、表、核心代码等形象化工具和手段，详细描述各个部件的关键实现。



➤ RF 单元及 MUX:

对 32 位指令 $inst$ 进行解析，得到读地址 $rR1(inst[19:15])$ 、 $rR2(inst[24:20])$ 和写地址 $wR(inst[11:7])$ 。寄存器的读没有时钟限制，因此可通过组合逻辑读出寄存器中数据 $rD1 = regfile[rR1]$ 、 $rD2 = regfile[rR2]$ 。寄存器的写与时钟上升沿同步，因此通过时序逻辑实现：当 CPU 复位时，各寄存器中数据均置为 0；如果写使能 we 为 1 且写入寄存器号不为 0，将写寄存器数据 wD 写入目标寄存器；如果写入寄存器号为 0，则 $x0$ 寄存器内数据始终为 0。

其中写寄存器数据 wD 由四选一多路选择器选择，其写入数据包括：运算器结果 $ALU.C$ 、存储器读出结果 $DRAM.rdo$ 、 $pc + 4$ 、扩展后的立即数 $SEXT.ext$ ，并由 rf_wsel 信号决定写入数据。

Name	I/O	Width	Description
clk	input	1	时钟信号
rst	input	1	复位信号
rR1	input	5	读地址1
rR2	input	5	读地址2
wR	input	5	写地址
from_alu	input	32	执行单元的计算后的数据
from_dram	input	32	数据寄存器取出的数据
from_imm	input	32	立即数扩展单元扩展后的立即数
from_pc4	input	32	$pc+4$
rf_wsel	input	2	控制信号：选择写回寄存器堆的数据来源
we	input	1	控制信号：写使能
rD1	output	32	读寄存器数据1
rD2	output	32	读寄存器数据2
wD	output	32	写寄存器数据

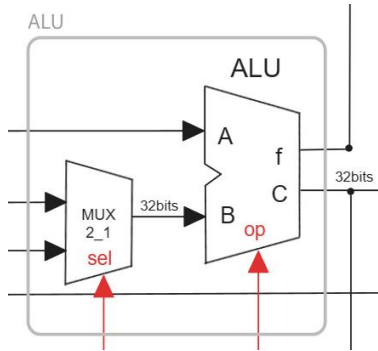
➤ SEXT 单元

立即数扩展单元，根据控制信号对指令中的立即数进行扩展。

Name	I/O	Width	Description
din	input	25	指令中的立即数 $inst[31:7]$
sext_op	input	3	控制扩展方式
ext	output	32	扩展后的立即数

```
always @(*) begin
  case(sext_op)
    `EXT_I: ext = din[24]? {20'hFFFF,din[24:13]} : {20'h0000,din[24:13]};
    `EXT_S: ext = din[24]? {20'hFFFF,din[24:18],din[4:0]} : {20'h0000,din[24:18],din[4:0]};
    `EXT_B: ext = din[24]? {20'hFFFF,din[0],din[23:18],din[4:1],1'b0} : {20'h0000,din[0],din[23:18],din[4:1],1'b0};
    `EXT_U: ext = {din[24:5],12'h000};
    `EXT_J: ext = din[24]? {12'hFFF,din[12:5],din[13],din[23:14],1'b0} : {12'h000,din[12:5],din[13],din[23:14],1'b0};
    `EXT_SHIFT: ext = {27'd0,din[17:13]};
    default: ext = 32'h0000_0000;
  endcase
end
```

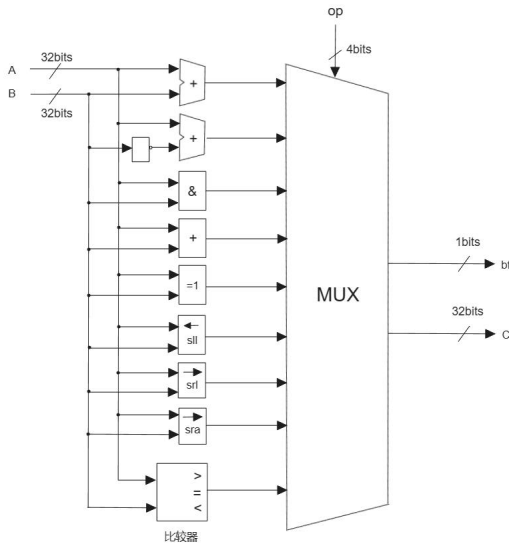
3. 执行模块



➤ ALU 单元及多路选择器:

根据控制信号 `alu_op` 选择运算方式并进行相应运算，其中包括加、减（补码）、按位与、按位或、按位异或、逻辑右移、逻辑左移、算术右移及比较。其中写入 ALU.B 端口的数据由二选一选择器进行选择，包括来自寄存器 `rR2` 的数据和来自立即数扩展单元 `SEXT` 扩展后的立即数。

Name	I/O	Width	Description
data1	input	32	操作数1
data2	input	32	操作数2（来自RF.rD2）
imm	input	32	操作数2（来自SEXT.ext）
sel	input	1	控制信号：选择ALU.B端口的数据来源
op	input	4	控制信号：选择运算方式
C	input	32	运算结果
bf	input	1	标志位



```
17 wire [31:0]A = data1;
18 wire [31:0]B = sel? imm : data2;
19 wire [4 :0]shamt = B[4:0];
20
21 always @(*) begin
22   case(op)
23     ADD: C = A+B;
24     SUB: C = A+(-B)+1;
25     AND: C = A & B;
26     OR: C = A | B;
27     XOR: C = A ^ B;
28     SLL: C = A << shamt;
29     SRL: C = A >> shamt;
30     SRA: C = $signed(A) >>> shamt;
31     BEQ: bf = (A==B)? 1'b1 : 1'b0;
32     BNE: bf = (A!=B)? 1'b1 : 1'b0;
33     BLT: begin
34       C = A+(-B)+1;
35       bf = (C[31])? 1'b1 : 1'b0;
36     end
37     BGE: begin
38       C = A+(-B)+1;
39       bf = (~C[31])? 1'b1 : 1'b0;
40     end
41     default: C = 32'd0;
42   endcase
43 end
```

4. 访存模块

➤ DRAM:

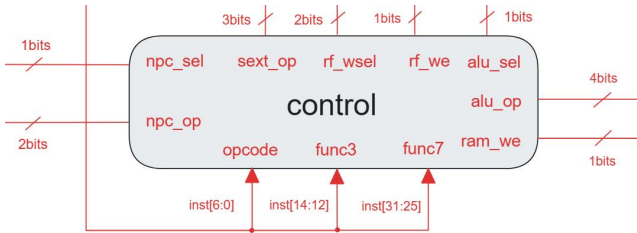
数据存储器，可读可写，用于存储数据。

Name	I/O	Width	Description
clk	input	1	时钟信号
adr	input	32	访存地址
rdo	output	32	读数据
we	input	1	写使能
wdin	input	32	写数据

5. 控制模块

对 32 位指令 inst 进行解析，得到操作码 opcode (inst [6:0])和功能码 func3(inst [14:12])、func7(inst[31:25])，产生指令执行所需的控制信号。控制信号包括功能选择信号和多路选择信号。

Name	I/O	Width	Description
inst	input	32	指令
bf	input	1	条件分支是否跳转
npc_op	output	2	控制NPC部件产生指令地址
rf_we	output	1	控制寄存器堆的读写操作
rf_wsel	output	2	控制寄存器堆写回数据的来源
sext_op	output	3	控制SEXT部件的符号扩展方式
alu_op	output	4	控制ALU的运算类型
alub_sel	output	1	控制ALU输入B的数据来源
ram_we	output	1	控制DRAM的读写操作



1.3 单周期 CPU 仿真及结果分析

要求：包含逻辑运算、访存、分支跳转三类指令的仿真截图及波形分析；每类指令的截图和分析中，至少包含 1 条具体指令；截图需包含信号名和关键信号。

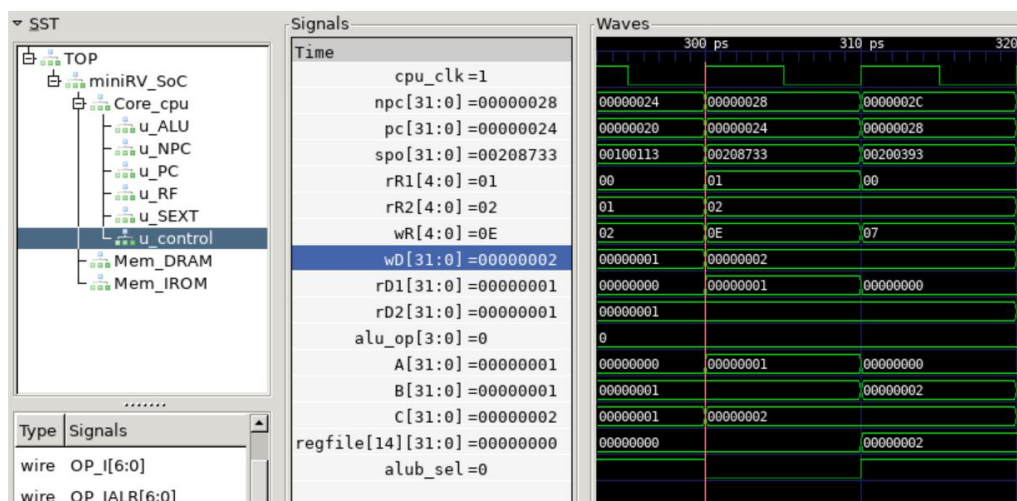
（由于单周期 CPU 一条完整指令在一个周期内执行完毕并，因此在此仅分析单个周期）

1. 逻辑运算

➤ add 指令：

```

0000001c <test_3>:
1c: 00100093      addi    x1,x0,1
20: 00100113      addi    x2,x0,1
24: 00208733      add     x14,x1,x2
28: 00200393      addi    x7,x0,2
2c: 00300193      addi    x3,x0,3
30: 4a771a63      bne     x14,x7,4e4 <fail>
  
```

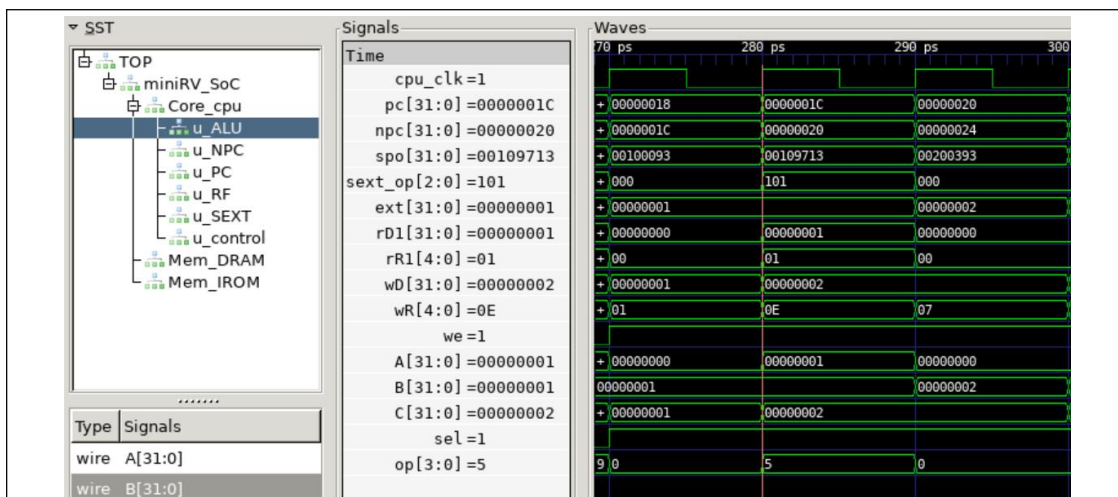


根据波形显示，当 pc=00000024 时，从指令存储器里取出指令 00208733，npc 输出下一条顺序执行的指令地址 00000028。将指令分解后将 rR1=x1、rR2=x1、wR=x14 传给寄存器堆，并从相应源寄存器取出数据 rD1=1、rD2=1。运算器执行加法运算（alu_op=4'b0000）得到结果 C=2，并将结果传回 wD、保存在 x14 寄存器中。

➤ slli 指令

```

00000018 <test_3>:
18: 00100093      addi    x1,x0,1
1c: 00109713      slli    x14,x1,0x1
20: 00200393      addi    x7,x0,2
24: 00300193      addi    x3,x0,3
28: 26771063      bne     x14,x7,288 <fail>
  
```



根据波形显示，当 `pc=0000001c` 时，从指令存储器里取出指令 `00109713`，npc 输出下一条顺序执行的指令地址 `00000020`。将指令分解后将 `rR1=x1`、`wR=x14` 传给寄存器堆，并从相应源寄存器取出数据 `rD1=1`。立即数扩展单元选择 SHIFT 立即数扩展方式

（`sext_op=101`）并赋值给 ALU.B 端口。运算器执行左移运算（`alu_op=4'b0000`）得到结果 `C=2`，并将结果传回 `wD`、保存在 `x14` 寄存器中。

2. 访存指令

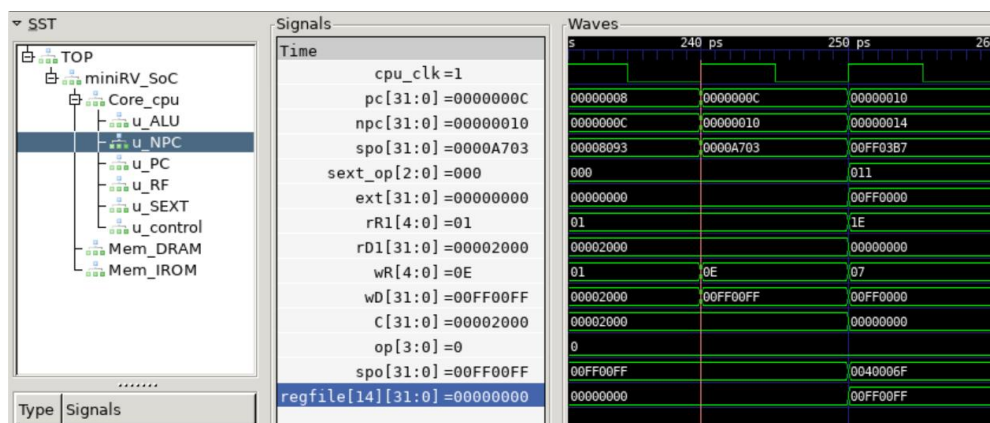
➤ lw 指令

`00000004 <reset_vector>:`

```

4: 000020b7          lui    x1,0x2
8: 00008093          addi   x1,x1,0 # 2000 <begin_signature>
c: 0000a703          lw     x14,0(x1)
10: 00ff03b7          lui    x7,0xff0
14: 0ff38393          addi   x7,x7,255 # ff00ff <_end+0xfe0ef>
18: 00200193          addi   x3,x0,2
1c: 26771a63          bne    x14,x7,290 <fail>

```



根据波形显示，当 `pc=0000000c` 时，从指令存储器里取出指令 `0000A703`，npc 输出下一条顺序执行的指令地址 `00000010`。将指令分解后将 `rR1=x1`、`wR=x14` 传给寄存器堆，并从相应源寄存器取出数据 `rD1=00002000`。立即数扩展单元选择 I 型立即数扩展方式

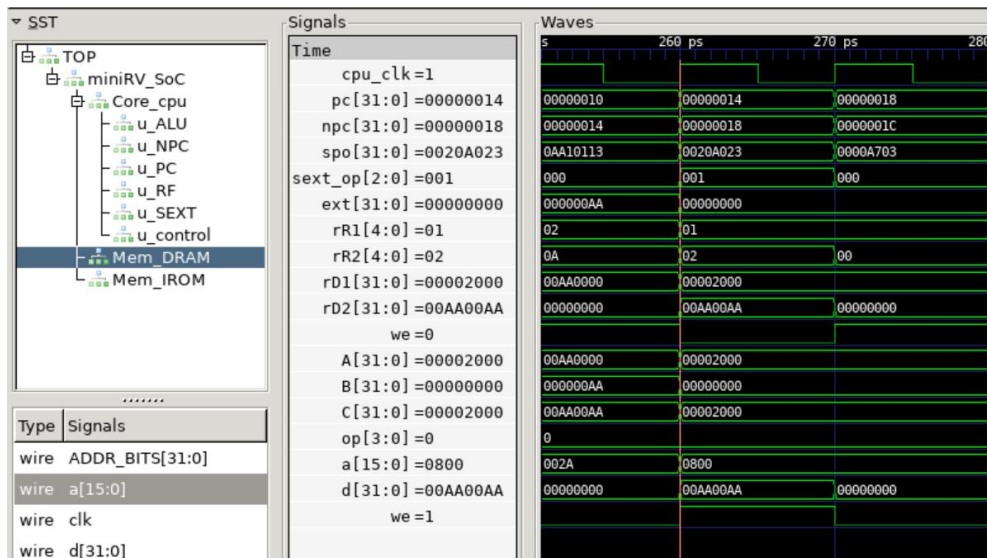
（`sext_op=000`）并赋值给 ALU.B 端口，运算器执行加法运算（`alu_op=4'b0000`）得到结果 `C=00002000`。结果 `C` 传至数据寄存器，从地址 `00002000` 取出数据 `00FF00FF` 并保存在 `x14` 寄存器中。

➤ sw 指令

```

00000004 <reset_vector>:
  4: 000020b7          lui      x1,0x2
  8: 00008093          addi     x1,x1,0 # 2000 <begin_signature>
  c: 00aa0137          lui      x2,0xaa0
 10: 0aa10113          addi     x2,x2,170 # aa00aa <_end+0xa9e07a>
 14: 0020a023          sw       x2,0(x1)
 18: 0000a703          lw       x14,0(x1)
 1c: 00aa03b7          lui      x7,0xaa0
 20: 0aa38393          addi     x7,x7,170 # aa00aa <_end+0xa9e07a>
 24: 00200193          addi     x3,x0,2
 28: 46771063          bne      x14,x7,488 <fail>

```



根据波形显示，当 $pc=00000014$ 时，从指令存储器里取出指令 $0020A023$ ，npc 输出一条顺序执行的指令地址 00000018 。将指令分解后将 $rR1=x1$ 、 $rR1=x2$ 传给寄存器堆，并从相应源寄存器取出数据 $rD1=00002000$ 、 $rD2=00AA00AA$ 。立即数扩展单元选择 S 型立即数扩展方式 ($sext_op=001$) 并赋值给 ALU.B 端口，运算器执行加法运算 ($alu_op=4'b0000$) 得到结果 $C=00002000$ 。地址 C 与数据 $rD2$ 传至数据寄存器，此时 DRAM 写使能为 1，即将 $rD2=00AA00AA$ 储存在数据存储器的地址 00002000 处。

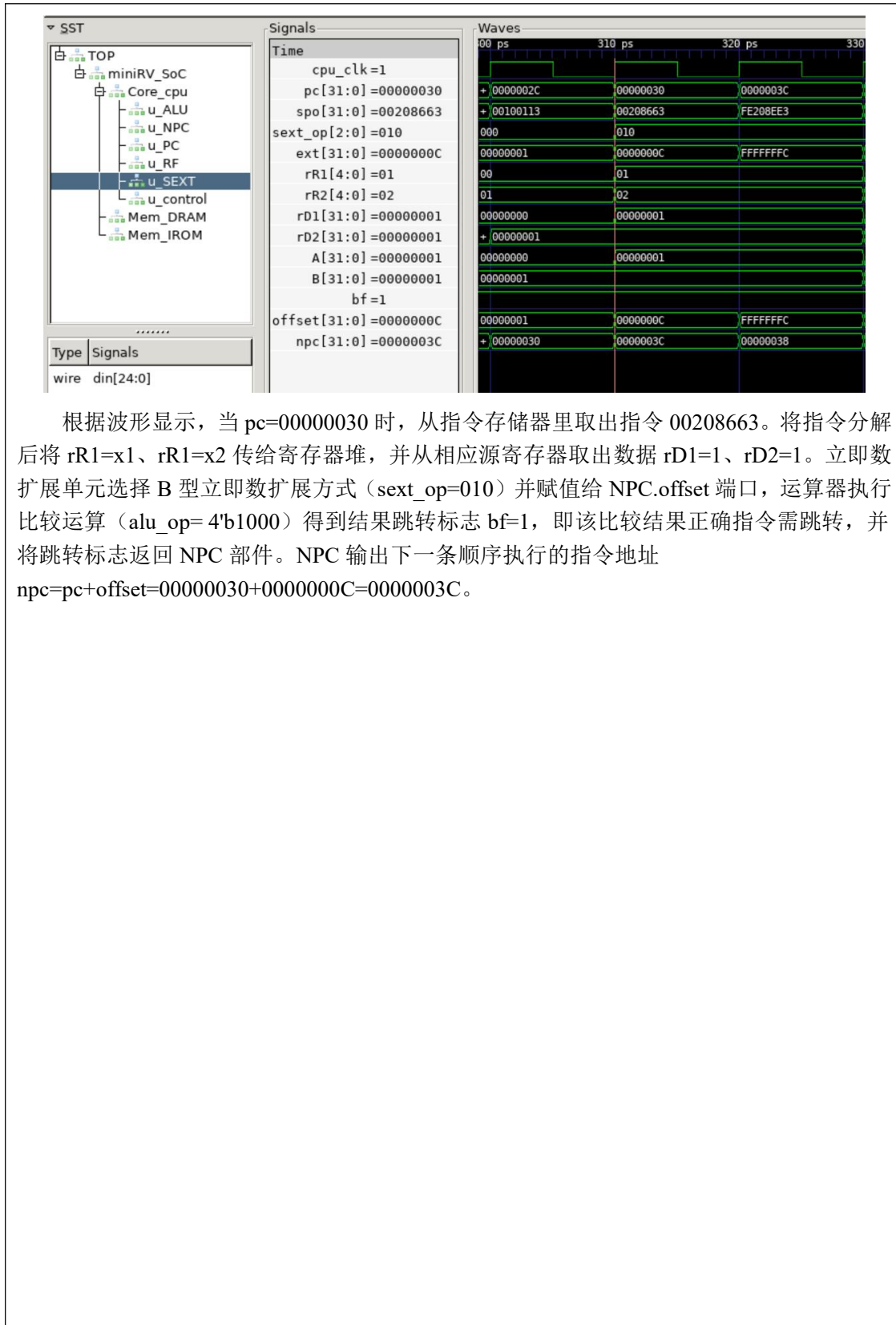
3. 分支跳转

➤ beq 指令

```

00000024 <test_3>:
 24: 00300193          addi     x3,x0,3
 28: 00100093          addi     x1,x0,1
 2c: 00100113          addi     x2,x0,1
 30: 00208663          beq      x1,x2,3c <test_3+0x18>
 34: 28301863          bne      x0,x3,2c4 <fail>
 38: 00301663          bne      x0,x3,44 <test_4>
 3c: fe208ee3          beq      x1,x2,38 <test_3+0x14>
 40: 28301263          bne      x0,x3,2c4 <fail>

```



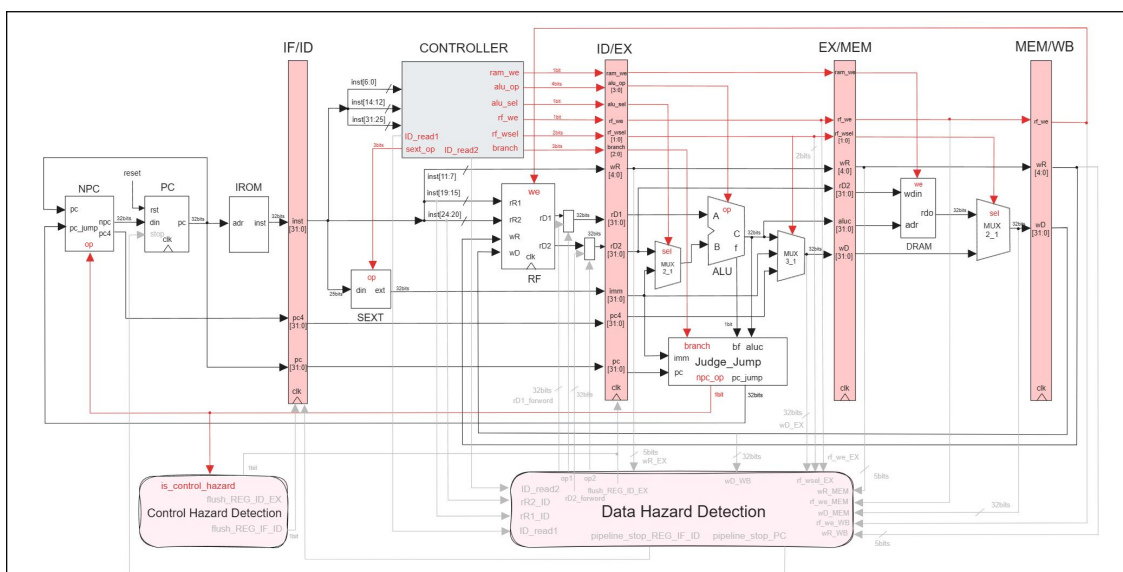
根据波形显示，当 $pc=00000030$ 时，从指令存储器里取出指令 00208663 。将指令分解后将 $rR1=x1$ 、 $rR1=x2$ 传给寄存器堆，并从相应源寄存器取出数据 $rD1=1$ 、 $rD2=1$ 。立即数扩展单元选择 B 型立即数扩展方式 ($sext_op=010$) 并赋值给 NPC.offset 端口，运算器执行比较运算 ($alu_op=4'b1000$) 得到结果跳转标志 $bf=1$ ，即该比较结果正确指令需跳转，并将跳转标志返回 NPC 部件。NPC 输出下一条顺序执行的指令地址 $npc=pc+offset=00000030+0000000C=0000003C$ 。

2 流水线 CPU 设计与实现

2.1 流水线 CPU 数据通路

要求：贴出完整的流水线数据通路图，无需画出模块内的具体逻辑，但要标出模块的接口信号名、模块之间信号线的信号名和位宽，并用文字阐述各模块的功能。

此外，数据通路图应当能体现出流水线是如何划分的，并用文字阐述每个流水级具备什么功能、需要完成哪些操作。



各级流水线功能:

1. 取指阶段 (IF)

- 根据当前 PC 值从指令存储器中读取一条指令，并更新下一条指令的 PC 值
- 产生冒险时，保持 PC 的值及 IF/ID 流水线寄存器不变

2. 译码阶段 (ID)

- 分解指令，从寄存器堆中取出操作数
- 得到对应类型扩展的立即数
- 控制器中产生控制信号
- 判断当前是否需要数据前递以及停顿

3. 执行阶段 (EX)

- 根据控制信号及操作数执行相应的运算、判断 B 型指令是否跳转
- 计算访存地址
- 通过多路选择器对写回寄存器堆的数据进行选择（也即 EX 阶段前递的数据）
- 得到 B 型（如需要跳转）/jal/jalr 型指令跳转到的 pc 值，包括 $pc+imm$ 、 $pc+4$ 、 $rs1+imm$ 三种情况，并返回给 IF 阶段的 NPC 部件

4. 访存阶段 (MEM)

- 根据控制信号以及地址对数据存储器执行相应的访存操作
- 通过多路选择器对写回寄存器堆的数据进行选择（也即 MEM 阶段前递的数据）

5. 写回阶段 (WB)

- 将执行阶段的结果或访存取出的数据写回寄存器堆

模块说明:

1. 取指模块

- PC: 存储当前指令地址的 32 位寄存器，指令执行时负责更新当前 PC 值（时序逻辑）
- NPC: 得到下一条指令的 PC 值（组合逻辑）
- IROM: 指令存储器，根据 PC 值得到对应的指令

2. 译码模块

- RF: 寄存器堆，可根据寄存器编号读出源寄存器数据或将数据写入目标寄存器（异步读（组合逻辑）、同步写（时序逻辑））
- SEXT: 立即数扩展单元，根据指令中立即数的格式进行扩展（组合逻辑）

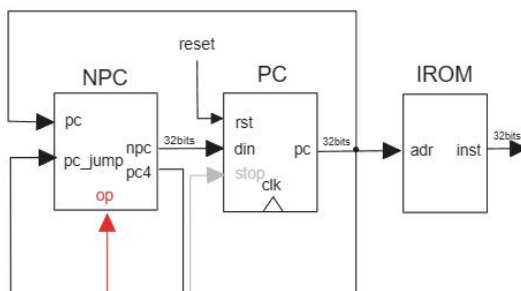
3. 执行模块

- MUX: 多路选择器, 选择输入 ALU.B 端口的数据
 - ALU: 运算器, 根据控制信号对输入数据执行加、减、与、或、异或、移位、比较等运算 (组合逻辑)
 - wD_MUX_1: 多路选择器, 对写回寄存器堆的数据进行第一次选择 (也即 EX 阶段前递的数据)
 - Judge_Jump: 得到 B 型/jal/jalr 型指令跳转到的 pc 值, 包括 pc+imm、pc+4、rs1+imm 三种情况, 并将是否跳转及跳转地址返回给 IF 阶段的 NPC 部件
4. 存储模块
- DRAM: 数据存储器, 可读可写, 用于存储数据
 - wD_MUX_2: 多路选择器, 选择写回寄存器堆的数据是执行阶段的结果还是访存取出的数据
5. 控制模块
- Control: 控制器, 根据指令中 opcode、func3、func7 的值得到各个器件的控制信号
6. 冒险检测器模块
- Data_HazardDetection: 检测并处理数据冒险, 发出前递、停顿、清除的控制信号和前递相关数据
 - Control_HazardDetection: 检测并处理控制冒险, 发出清除的控制信号
7. 流水线寄存器模块
- REG_IF_ID: IF 和 ID 之间的流水线寄存器
 - REG_ID_EX: ID 和 EX 之间的流水线寄存器
 - REG_EX_MEM: EX 和 MEM 之间的流水线寄存器
 - REG_MEM_WB: MEM 和 WB 之间的流水线寄存器

2.2 流水线 CPU 模块详细设计

要求: 以表格的形式列出所有与单周期不同的部件的接口信号、位宽、功能描述等, 并结合图、表、核心代码等形象化工具和手段, 详细描述这些部件的关键实现。此外, 如果实现了冒险控制, 必须结合数据通路图, 详细说明数据冒险、控制冒险的解决方法。

➤ 取指模块



➤ PC 单元:

指令正常执行时, pc 更新为下一条指令地址 din; CPU 复位时, 寄存器初始化为 0 (在单周期 trace 时需要设置标志位在初始时停滞一个周期); 当停顿信号 pipeline_stop 为 1 时, 保持 pc 内容在下一个时钟周期不变。

Name	I/O	Width	Description
clk	input	1	时钟信号
rst	input	1	复位信号
pipeline_stop	input	1	停顿信号
din	input	32	下一条指令地址
pc	output	32	当前指令地址

➤ NPC 单元:

NPC 具有两个用途:

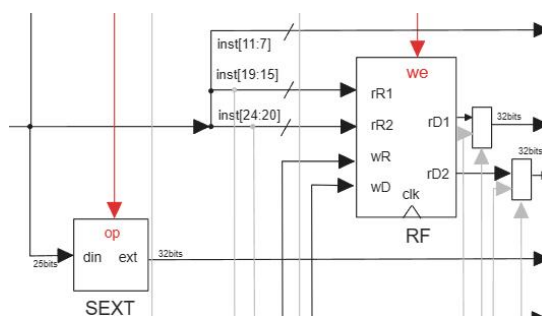
- pc4 输出 pc+4 的值
- npc 输出下一条指令地址, 由 npc_op 控制信号决定, 共分为两种情况:
 - 当 npc_op 为 0 时: $\text{npc} \leftarrow \text{pc}+4$
 - 当 npc_op 为 1 时: $\text{npc} \leftarrow \text{pc_jump}$ (执行阶段计算出的跳转到的地址)

Name	I/O	Width	Description
npc_op	input	1	控制跳转信号
pc	input	32	当前指令地址
pc_jump	input	32	跳转到地址
pc4	output	32	pc+4
npc	output	32	下一条指令地址

➤ IROM 单元:

指令存储器, 根据 PC 值得到对应的指令, 同单周期。

2. 译码模块



➤ RF 单元:

对 32 位指令 $inst$ 进行解析, 得到读地址 $rR1(inst[19:15])$ 、 $rR2(inst[24:20])$ 和写地址 $wR(inst[11:7])$ 。寄存器的读没有时钟限制, 因此可通过组合逻辑读出寄存器中数据 $rD1 = regfile[rR1]$ 、 $rD2 = regfile[rR2]$ 。寄存器的写与时钟上升沿同步, 因此通过时序逻辑实现: 当 CPU 复位时, 各寄存器中数据均置为 0; 如果写使能 we 为 1 且写入寄存器号不为 0, 将写寄存器数据 wD 写入目标寄存器; 如果写入寄存器号为 0, 则 $x0$ 寄存器内数据始终为 0。

Name	I/O	Width	Description
clk	input	1	时钟信号
rst	input	1	复位信号
rR1	input	5	读地址1
rR2	input	5	读地址2
wR	input	5	写地址
wD	input	32	写寄存器数据
we	input	1	控制信号: 写使能
rD1	output	32	读寄存器数据1
rD2	output	32	读寄存器数据2

➤ SEXT 单元

立即数扩展单元, 根据控制信号对指令中的立即数进行扩展, 同单周期。

Name	I/O	Width	Description
dIn	input	25	指令中的立即数 $inst[31:7]$
sext_op	input	3	控制扩展方式
ext	output	32	扩展后的立即数

3. 执行模块

➤ ALU 单元: 同单周期

➤ wD_MUX_1:

写回寄存器堆的数据 wD 的第一次选择。因为 $DRAM.rdo$ 在访存阶段才能得到, 因此此处的 wD 的从 $ALU.C$ 、 $SEXT.ext$ 、和 $NPC.pc4$ 中选择其一, 同时, 此处选择后得到的 wD 也是 EX 阶段前递的数据

Name	I/O	Width	Description
aluc_in	input	32	运算器结果 (ALU.C)
sext_in	input	32	扩展后立即数 (SEXT.ext)
pc4_in	input	32	$NPC.pc4$
wD	output	32	写寄存器数据
rf_wsel	input	2	控制信号: 选择写寄存器数据

➤ Judge_Jump: 根据控制信号 $branch$ 得到跳转标志 npc_op 和跳转到的地址 pc_jump , 其

中 pc_jump 包括 pc+imm、pc+4、rs1+imm 三种情况。并将二者返回给 IF 阶段的 NPC 部件

Name	I/O	Width	Description
branch	input	3	控制信号:{B,jal,jalr}
bf	input	1	条件分支是否跳转
pc	input	32	当前指令地址
imm	input	32	扩展后立即数
aluc	input	32	运算器所得结果
nop_op	output	1	跳转标志
pc_jump	output	32	跳转到的指令地址

```

always @(*) begin
    if(branch[2] & bf)      pc_jump = pc+imm;      //是B型指令且跳转
    else if(branch[1])      pc_jump = pc+imm;      //JAL
    else if(branch[0])      pc_jump = aluc;        //JALR
    else                    pc_jump = pc+32'd4;
end

always @(*) begin
    if(branch[2] & bf)      npc_op = 1'b1;          //是B型指令且跳转
    else if(branch[2] & !bf) npc_op = 1'b0;          //是B型指令但不跳转
    else if(branch[1])      npc_op = 1'b1;          //JAL
    else if(branch[0])      npc_op = 1'b1;          //JALR
    else                    npc_op = 1'b0;
end

```

4. 访存模块

➤ DRAM: 数据存储器，同单周期

➤ wD_MUX_1:

写回寄存器堆的数据 wD 的第二次选择。选择写回寄存器堆的数据是执行阶段的结果还是访存取出的数据。

Name	I/O	Width	Description
wD_in	input	32	执行阶段的结果
dram_in	input	32	访存取出的数据
wD	output	32	写寄存器数据
rf_wsel	input	2	控制信号: 选择写寄存器数据

5. 控制模块

对 32 位指令 inst 进行解析，得到操作码 opcode (inst [6:0])和功能码 func3(inst [14:12])、func7(inst[31:25])，产生指令执行所需的控制信号。控制信号包括功能选择信号和多路选择信号。在单周期的基础之上删除了 npc_op，增加了 ID_read1、ID_read2、branch 信号，其定义及实现如下。

Name	I/O	Width	Description
inst	input	32	指令
ID_read1	output	1	RS1寄存器的读标志信号
ID_read2	output	1	RS2寄存器的读标志信号
branch	output	3	控制信号:{B,jal,jalr}
rf_we	output	32	控制寄存器堆的读写操作
rf_wsel	output	2	控制寄存器堆写回数据的来源
sext_op	output	3	控制SEXT部件的符号扩展方式
alu_op	output	4	控制ALU的运算类型
alub_sel	output	1	控制ALU输入口B的数据来源
ram_we	output	1	控制DRAM的读写操作
have_inst	output	1	当前时钟周期是否有指令执行到EX阶段

```

assign ID_read1 = (opcode == OP_R) | (opcode == OP_LW) | (opcode == OP_JALR) | (opcode == OP_I) | (opcode == OP_SW) | (opcode == OP_B);
assign ID_read2 = (opcode == OP_R) | (opcode == OP_LW) | (opcode == OP_JALR) | (opcode == OP_SW) | (opcode == OP_B);

wire is_B = (opcode == OP_B)? 1'b1 : 1'b0;
wire is_JAL = (opcode == OP_J)? 1'b1 : 1'b0;
wire is_JALR = (opcode == OP_JALR)? 1'b1 : 1'b0;
assign branch = {is_B, is_JAL, is_JALR};

```

6. 冒险检测器模块

- **Data_HazardDetection:** 检测并处理数据冒险，发出前递、停顿、清除的控制信号和前递相关数据。前递逻辑的实现位于 ID/EX 流水线寄存器中。

Name	I/O	Width	Description
ID_read1、ID_read2	input	1	RS1/2寄存器的读标志信号
rR1_ID、rR2_ID	input	5	ID阶段源寄存器号
rf_wsel	input	2	控制寄存器堆写回数据的来源
wR_EX、wR_MEM、wR_WB	input	5	各阶段写地址
wD_EX、wD_MEM、wD_WB	input	32	各阶段写寄存器数据
rf_we_EX、rf_we_MEM、rf_we_WB	input	1	各阶段写使能
forward_op1、forward_op1	output	1	rdata1/2是否选择前递数据
rD1_forward、rD1_forward	output	32	rdata1/2前递数据
pipeline_stop_PC	output	1	PC停顿信号
pipeline_stop_REG_IF_ID	output	1	IF/ID停顿信号
flush_REG_ID_EX	output	1	ID/EX清除信号

检测代码如下：

```

/**
 * 数据冒险检测：核心条件：EX/MEM/WB_wR==rR1/rR2；附加的条件：此阶段RF可写，rD1/rD2必须被使用，wR不能为x0。
 */

//RAW-A
wire RAW_A_rR1 = (wR_EX == rR1_ID) && rf_we_EX && ID_read1 && (wR_EX != 5'b0);
wire RAW_A_rR2 = (wR_EX == rR2_ID) && rf_we_EX && ID_read2 && (wR_EX != 5'b0);

//RAW-B
wire RAW_B_rR1 = (wR_MEM == rR1_ID) && rf_we_MEM && ID_read1 && (wR_MEM != 5'b0);
wire RAW_B_rR2 = (wR_MEM == rR2_ID) && rf_we_MEM && ID_read2 && (wR_MEM != 5'b0);

//RAW-C
wire RAW_C_rR1 = (wR_WB == rR1_ID) && rf_we_WB && ID_read1 && (wR_WB != 5'b0);
wire RAW_C_rR2 = (wR_WB == rR2_ID) && rf_we_WB && ID_read2 && (wR_WB != 5'b0);

//load_use
wire load_use_hazard = (RAW_A_rR1 || RAW_A_rR2) && (rf_wsel == `WB_DREM);

```

解决方案：停顿+前递

```
//判断是否需要前递:RAW任意冒险出现即需要前递
assign forward_op1 = RAW_A_rR1 | RAW_B_rR1 | RAW_C_rR1;
assign forward_op2 = RAW_A_rR2 | RAW_B_rR2 | RAW_C_rR2;

//load_use冒险需要特殊处理: 停顿(PC及IF/ID不变+ID/EX置0)+前递
always @ (*) begin
    if(load_use_hazard) begin
        pipeline_stop_PC = 1'b1;
        pipeline_stop_REG_IF_ID = 1'b1;
        flush_REG_ID_EX = 1'b1;
    end
    else begin
        pipeline_stop_PC = 1'b0;
        pipeline_stop_REG_IF_ID = 1'b0;
        flush_REG_ID_EX = 1'b0;
    end
end

//计算前递数据
always @ (*) begin
    if (RAW_A_rR1) rD1_forward = wD_EX;
    else if (RAW_B_rR1) rD1_forward = wD_MEM;
    else if (RAW_C_rR1) rD1_forward = wD_WB;
    else rD1_forward = 32'b0;
end

always @ (*) begin
    if (RAW_A_rR2) rD2_forward = wD_EX;
    else if (RAW_B_rR2) rD2_forward = wD_MEM;
    else if (RAW_C_rR2) rD2_forward = wD_WB;
    else rD2_forward = 32'b0;
end
```

- **Control_HazardDetection**: 检测并处理控制冒险，发出清除的控制信号。在此将 jal 和 jalr 这两种必跳转指令也视为“控制冒险”，跳转从 ID 阶段延后至 EX 阶段。

Name	I/O	Width	Description
is_control_hazard	input	1	是否产生控制冒险
flush_REG_ID_EX	output	1	ID/EX流水线寄存器清除信号
flush_REG_IF_ID	output	1	IF/ID流水线寄存器清除信号

```
module Control_HazardDetection(
    input wire is_control_hazard, //1为产生控制冒险

    //清除信号
    output wire flush_REG_ID_EX,
    output wire flush_REG_IF_ID
);

//静态分支预测:总是预测不跳转，一旦预测错误即清除flush IF/ID, ID/EX
assign flush_REG_ID_EX = is_control_hazard? 1'b1 :1'b0;
assign flush_REG_IF_ID = is_control_hazard? 1'b1 :1'b0;

endmodule
```

7. 流水线寄存器模块

流水线寄存器	保存的信号	其余信号/功能
REG_IF_ID	pc, pc4, inst	pipeline_stop, flush
REG_ID_EX	pc, pc4, imm, rD1, rD2, wR, have_inst, rf_wsel, rf_we, alu_op, alub_sel, ram_we, branch,	flush, 前递相关信号及数据(forward_op1, forward_op2, rD1_forward, rD2_forward)
REG_EX_MEM	wD, wR, rD2, pc, aluc, have_inst, rf_wsel, rf_we, ram_we	当前指令地址
REG_MEM_WB	wD, wR, pc, have_inst, rf_we,	扩展后立即数

其中前递逻辑在 REG_ID_EX 寄存器中的实现代码如下：

```
always @(posedge clk or posedge rst) begin
    if(rst) rD1_out <= 32'b0;
    else if(forward_op1) rD1_out <= rD1_forward; //前递
    else rD1_out <= rD1_in;
end

always @(posedge clk or posedge rst) begin
    if(rst) rD2_out <= 32'b0;
    else if(forward_op2) rD2_out <= rD2_forward; //前递
    else rD2_out <= rD2_in;
end
```

2.3 流水线 CPU 仿真及结果分析

要求：包含控制冒险和数据冒险三种情形的仿真截图，以及波形分析。若仅实现了理想流水，则此处贴上理想流水的仿真截图及详细的波形分析。

数据冒险

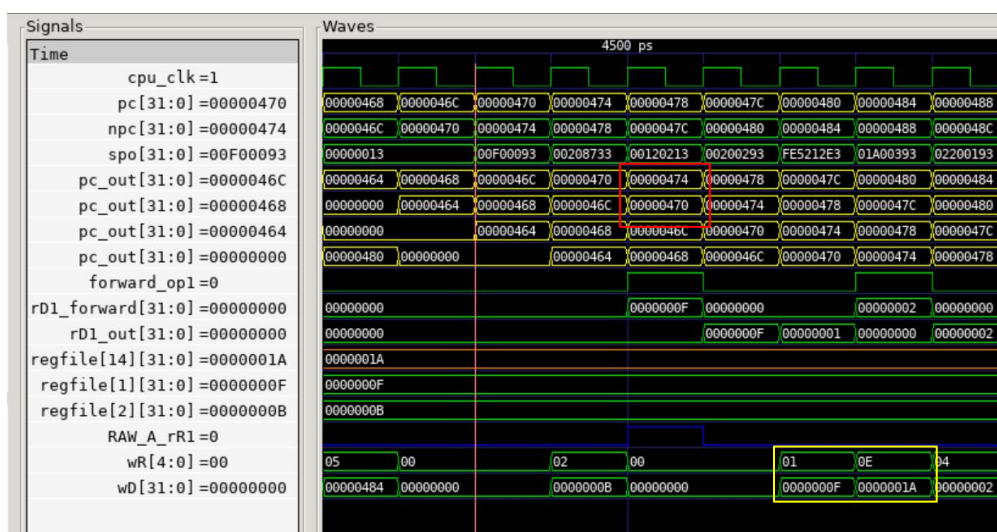
- **RAW-A**（其中 pc=470 和 pc=474 均与 x1 有关，产生数据冒险）

▣ 00000460 <test_34>:

```

460: 00000213      addi    x4,x0,0
464: 00b00113      addi    x2,x0,11
468: 00000013      addi    x0,x0,0
46c: 00000013      addi    x0,x0,0
470: 00f00093      addi    x1,x0,15
474: 00208733      add     x14,x1,x2
478: 00120213      addi    x4,x4,1 # 1 <_start+0x1>

```



由波形图可见，当 pc = 00000470 指令的 WB 阶段结束后，x1 寄存器中的值变为 0000_000F；但在下一条指令 pc=00000474 在 EX 阶段就需要用到 x1 寄存器内的值，但此时 x1 的值仍为原值而未改变，即 rs1 产生 RAW-A 数据冒险。对此，使用前递解决冒险。

当 pc = 00000470 指令位于 EX 阶段、pc=00000474 指令位于 ID 阶段时，数据冒险检测器检测到冒险产生（RAW_A_rR1=1），并对 rD1 进行前递：ID/EX 寄存器内，在前递信号 forward_op1=1 的控制下，输出的 rD1_out 变为 0000_000F 并进行后续运算。最终当 pc=00000474 指令位于 WB 阶段时，x14 寄存器内存入数据 0000001A，符合预期。

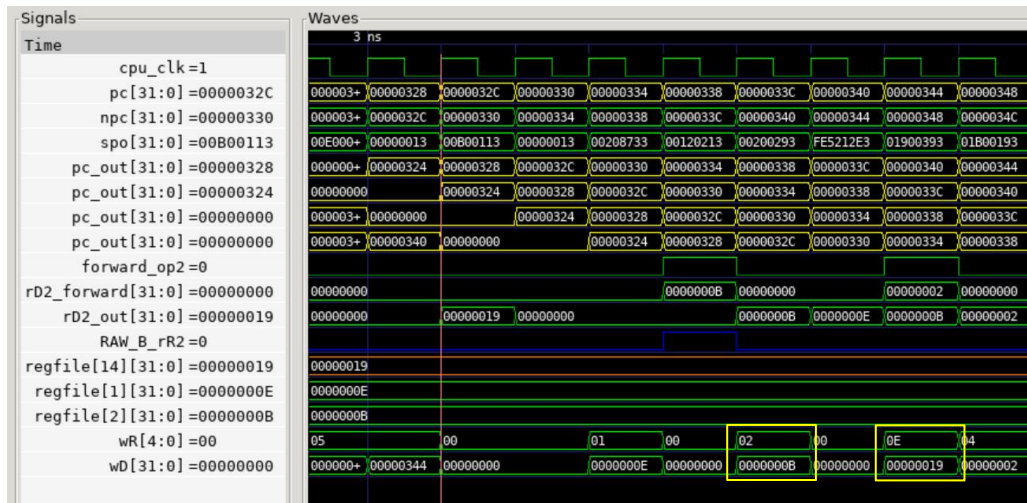
- **RAW-B**（其中 pc=32c 和 pc=334 均与 x2 有关，产生数据冒险）

▣ 00000320 <test_27>:

```

320: 00000213      addi    x4,x0,0
324: 00e00093      addi    x1,x0,14
328: 00000013      addi    x0,x0,0
32c: 00b00113      addi    x2,x0,11
330: 00000013      addi    x0,x0,0
334: 00208733      add     x14,x1,x2
338: 00120213      addi    x4,x4,1 # 1 <_start+0x1>

```



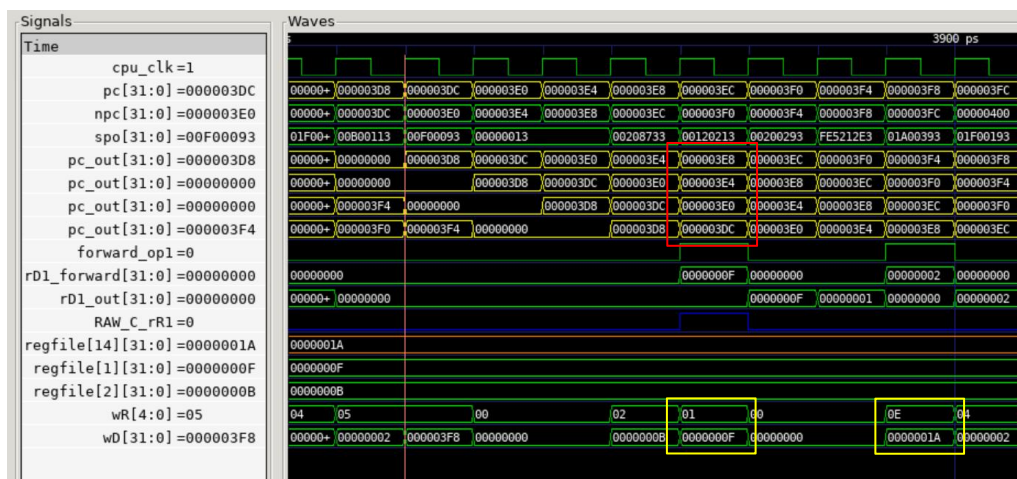
由波形图可见，当 $pc = 0000032C$ 指令的 WB 阶段结束后，x2 寄存器中的值变为 0000_000B；在下一条指令 $pc=00000334$ 在 EX 阶段就需要用到 x2 寄存器内的值，但此时 x2 的值仍为原值而未改变，即 rs 产生 RAW-B 数据冒险。对此，使用前递解决冒险。

当 $pc = 0000032C$ 指令位于 MEM 阶段、 $pc=00000334$ 指令位于 ID 阶段时，数据冒险检测器检测到冒险产生（ $RAW_B_rR2=1$ ），并对 rD2 进行前递：ID/EX 寄存器内，在前递信号 $forward_op2=1$ 的控制下，输出的 rD2_out 变为 0000_000B 并进行后续运算。最终当 $pc=00000334$ 指令位于 WB 阶段时，x14 寄存器内存入数据 00000019，符合预期。

➤ RAW-C（其中 $pc=3dc$ 和 $pc=3e8$ 均与 x1 有关，产生数据冒险）

▣ 000003d4 <test_31>:

3d4:	00000213	addi	x4,x0,0
3d8:	00b00113	addi	x2,x0,11
3dc:	00f00093	addi	x1,x0,15
3e0:	00000013	addi	x0,x0,0
3e4:	00000013	addi	x0,x0,0
3e8:	00208733	add	x14,x1,x2
3ec:	00120213	addi	x4,x4,1 # 1 <_start+0x1>



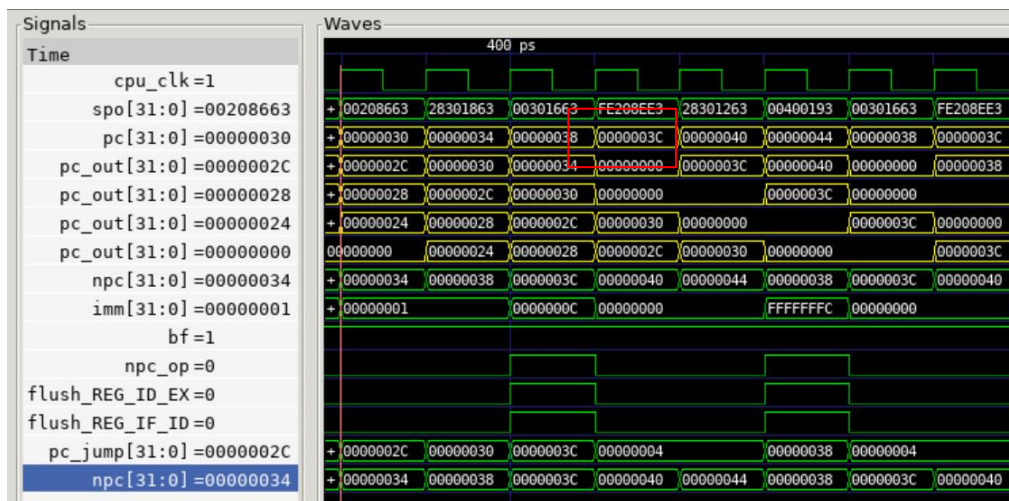
由波形图可见，当 $pc = 000003dc$ 指令的 WB 阶段结束后，x1 寄存器中的值变为 0000_000F；指令 $pc=000003e8$ 在 EX 阶段就需要用到 x1 寄存器内的值，但此时 x1 的

值仍为原值而未改变，即 rs1 产生 RAW-C 数据冒险。对此，使用前递解决冒险。

当 pc = 000003dc 指令位于 WB 阶段、pc=000003e8 指令位于 ID 阶段时，数据冒险检测器检测到冒险产生 (RAW_C_rR1=1)，并对 rD1 进行前递：ID/EX 寄存器内，在前递信号 forward_op1=1 的控制下，输出的 rD1_out 变为 0000_000F 并进行后续运算。最终当 pc=000003e8 指令位于 WB 阶段时，x14 寄存器内存入数据 0000001A，符合预期。

控制冒险

```
00000024 <test_3>:
24: 00300193          addi    x3,x0,3
28: 00100093          addi    x1,x0,1
2c: 00100113          addi    x2,x0,1
30: 00208663          beq     x1,x2,3c <test_3+0x18>
34: 28301863          bne     x0,x3,2c4 <fail>
38: 00301663          bne     x0,x3,44 <test_4>
3c: fe208ee3          beq     x1,x2,38 <test_3+0x14>
40: 28301263          bne     x0,x3,2c4 <fail>
```



对于控制冒险，在此采用静态分支预测，并总是预测不跳转。

由波形图显示，当 pc=00000030 指令位于 EX 阶段时，运算器进行比较运算，且由于 x1=x2=1，条件成立进行跳转（即 bf=1）。此时冒险检测器检测到发生控制冒险 (npc_op=1)，并清除后二条指令，即将信号 flush_REG_IF_ID 与 flush_REG_ID_EX 均置为 1。同时 Judge_Jump 模块根据 bf 信号计算出跳转到的地址 pc=0000003c 并赋值给 NPC.npc，最终下一条指令的 pc 为 0000003c。

3 设计过程中遇到的问题及解决方法

要求:包括设计过程中遇到的有价值的错误,或测试过程中遇到的有价值的问题。所谓有价值,指的是解决该错误或问题后,能够学到新的知识和技巧,或加深对已有知识的理解和运用。

1. 流水线布局与连线

流水线不同于单周期 CPU,需要在设计初始情况时考虑数据前递以及停顿的问题,因此在设计流水线 CPU 时调整了单周期 CPU 的布局,着重调整了 NPC 模块,并设置了专门计算跳转地址的 Judge_Jump 模块。但代价是在 EX 阶段才能获得跳转地址,相比于 jalr 和 jal 在 ID 就能获得跳转地址浪费了一个时钟周期。

```
always @(*) begin
    if(branch[2] & bf)      pc_jump = pc+imm;      //是B型指令且跳转
    else if(branch[1])      pc_jump = pc+imm;      //JAL
    else if(branch[0])      pc_jump = aluc;        //JALR
    else                    pc_jump = pc+32'd4;
end

always @(*) begin
    if(branch[2] & bf)      npc_op = 1'b1;          //是B型指令且跳转
    else if(branch[2] & !bf) npc_op = 1'b0;          //是B型指令但不跳转
    else if(branch[1])      npc_op = 1'b1;          //JAL
    else if(branch[0])      npc_op = 1'b1;          //JALR
    else                    npc_op = 1'b0;
end
```

2. 连线冲突

由于在 myCPU 中需要实例化多个模块,且一根信号线可能由多个其他信号决定,所以可能会导致一个输入对应多个输出的情况,造成连线冲突。在这种情况下,需要在 muCPU 模块中进行特殊处理。

```
/**HazardDetection***/

assign flush_REG_ID_EX = flush_REG_ID_EX_Control | flush_REG_ID_EX_Data;

Control_HazardDetection u_Control_HazardDetection(
    //input
    .is_control_hazard(npc_op),
    //output
    .flush_REG_ID_EX(flush_REG_ID_EX_Control),
    .flush_REG_IF_ID(flush_REG_IF_ID)
);

Data_HazardDetection u_Data_HazardDetection(
    //input
    .rR1_ID(inst_ID[19:15]),
    .rR2_ID(inst_ID[24:20]),
    .ID_read1(ID_read1),
    .ID_read2(ID_read2),
    .rf_wsel(rf_wsel_EX),
```

4 总结

要求：谈谈学完本课程后的个人收获以及对本课程的建议和意见。请在认真总结和思考后填写总结。

关于 CPU 的知识已经在计算机组成原理的课程上学过，但真正实践的时候需要注意的细节更多、更复杂。计算机设计与实践这门课程极大的巩固并增加了我对于 RISC-V CPU 的认识，也更加透彻的了解了流水线，这是仅学习理论知识所无法得到的宝贵经验。

硬件设计，图表先行。这次实验课我深刻的体会到画数据通路图的重要性，尽管每张图耗时不少，但为我代码的撰写以及后期的修改提供了极大的便利。同时，CPU 的设计也增加了我对 Verilog 语言的掌握。学习至今已经踩遍了无数 Verilog 语法的坑，现在编写相关代码时已经基本能够做到避开常见的语法错误，进行相对系统的编程。

当然，下板的过程相当折磨——因为没有便捷的调试方式，无法判断到底哪个地方发生了错误，只能从错误结果倒推猜测，这在流水线 CPU 的实验中体现的尤为明显，在实验室坐了整整一天才找到 bug 所在的痛苦实在不想回忆。当然，这其实也证明 trace 的特殊情况测试不全，如果能在 trace 时便检测出代码中存在的错误，下板的时候便不至于如此痛苦且低效。

最后，衷心感谢薛睿老师，不厌其烦的解答我的疑问。祝这门课程越办越好！